421. Maximum XOR of Two Numbers in an Array

```c
#include <stdlib.h>

struct TrieNode {
    struct TrieNode* children[2];
};
```

```c
struct TrieNode* createNode() {
    struct TrieNode* node = (struct TrieNode*)malloc(sizeof(struct TrieNode));
    node->children[0] = NULL;
    node->children[1] = NULL;
    return node;

}
```

```c
void insert(struct TrieNode* root, int num) {
    struct TrieNode* curr = root;
    for (int i = 31; i >= 0; i--) {
        int bit = (num >> i) & 1;
        if (!curr->children[bit]) {
            curr->children[bit] = createNode();
        }
        curr = curr->children[bit];
    }
}
```

```c
int getMaxXOR(struct TrieNode* root, int num) {
    struct TrieNode* curr = root;
    int maxXOR = 0;
    for (int i = 31; i >= 0; i--) {
        int bit = (num >> i) & 1;
        int toggledBit = 1 - bit;
        if (curr->children[toggledBit]) {
            maxXOR |= (1 << i);
            curr = curr->children[toggledBit];
        } else {
            curr = curr->children[bit];
        }
    }
    return maxXOR;
}
```

```c
void freeTrie(struct TrieNode* root) {
    if (!root) return;
    freeTrie(root->children[0]);
    freeTrie(root->children[1]);
    free(root);
}
```

```c
int findMaximumXOR(int* nums, int numsSize) {
    if (numsSize < 2) return 0;

    struct TrieNode* root = createNode();
    for (int i = 0; i < numsSize; i++) {
        insert(root, nums[i]);
    }

    int globalMax = 0;
    for (int i = 0; i < numsSize; i++) {
        int currentMax = getMaxXOR(root, nums[i]);
```

```
        if (currentMax > globalMax) {
            globalMax = currentMax;
        }
    }

    freeTrie(root);
    return globalMax;
}
```

476. Number Complement

```
int findComplement(int num) {
    unsigned int mask = ~0;
    while (num & mask) {
        mask <<= 1;
    }
    return num ^ ~mask;
}
```